

My Password CLI

Générateur et auditeur de mots de passe en Java 21 / Docker

1. Analyse fonctionnelle

My Password CLI est un outil en ligne de commande qui répond à deux besoins complémentaires : générer des mots de passe robustes selon des critères choisis par l'utilisateur, et évaluer objectivement la solidité réelle de ces mots de passe en s'appuyant sur un outil d'audit externe exécuté dans un conteneur Docker isolé.

1.1 Modes d'interaction

L'application propose deux modes, conformément au cahier des charges :

- Mode CLI paramètre : tous les choix sont passés en arguments (--length, --upper, --lower, --digits, --symbols, --count, --no-server).
- Mode interactif : si aucun argument n'est fourni, le programme pose les questions une à une au clavier (longueur, types de caractères, nombre de mots de passe, activation de l'audit Docker).

1.2 Paramètres de génération

Paramètre	Rôle
--length=N	Longueur du mot de passe (4 à 128 caractères)
--upper / --no-upper	Inclusion des lettres majuscules
--lower / --no-lower	Inclusion des lettres minuscules
--digits / --no-digits	Inclusion des chiffres
--symbols / --no-symbols	Inclusion des symboles spéciaux
--count=N	Mode rafale : nombre de mots de passe générés en une exécution (1 à 1000)
--no-server	Désactive l'appel au conteneur Docker (estimation locale de secours)

1.3 Indicateur de force

Pour chaque mot de passe généré, le programme affiche un niveau de robustesse parmi les cinq niveaux demandés : Très faible, Faible, Moyen, Fort, Très fort, ainsi qu'une estimation lisible du temps nécessaire pour le casser (par exemple "3 heures" ou "des siècles"), fournie par l'outil d'audit externe.

2. Analyse technique

2.1 Structure du programme Java

Le code Java est organisé en trois paquets aux responsabilités clairement séparées :

- cli : analyse des arguments de la ligne de commande et mode interactif (ArgumentParser, CliOptions).
- generator : génération cryptographiquement sûre des mots de passe via SecureRandom (PasswordGenerator, CharacterSet).
- strength : communication avec le conteneur Docker et estimation de secours (StrengthClient, StrengthLevel, StrengthResult, LocalStrengthEstimator).

La classe Main orchestre l'enchaînement : lecture des options, génération des mots de passe (éventuellement en rafale), puis audit de chacun via le conteneur.

2.2 Choix de l'outil de validation externe

Le score de robustesse n'est pas calculé par une simple règle Java (longueur + présence de catégories de caractères), ce qui serait peu pertinent face à de vrais mots de passe faibles mais "complexes en apparence" (ex : Azerty123!). Le projet s'appuie sur zxcvbn, l'algorithme open-source développé par Dropbox, reconnu pour estimer la robustesse en détectant les motifs réels d'attaque : mots du dictionnaire, dates, séquences de clavier, répétitions, substitutions l33t-speak, etc. Le score zxcvbn (0 à 4) correspond exactement aux cinq niveaux demandés dans le cahier des charges.

2.3 Architecture d'interopérabilité Java <-> Docker

zxcvbn est exposé par un micro-service Python (Flask) exécuté dans un conteneur Docker dédié (dossier docker/strength-checker). Ce conteneur écoute sur le port 5000 et expose deux routes :

- GET /health : vérification que le service est démarré (healthcheck Docker).
- POST /check : reçoit {"password": "..."} et retourne {"score": 0-4, "crack_time_display": "..."}.

Côté Java, la classe StrengthClient utilise exclusivement l'API standard java.net.http.HttpClient (disponible depuis Java 11, donc nativement en Java 21) pour envoyer une requête HTTP POST au conteneur et parser la réponse JSON. Aucune dépendance externe (Jackson, OkHttp...) n'est nécessaire.

Ce choix d'une API REST HTTP plutôt qu'un appel direct ("docker exec") présente plusieurs avantages DevOps :

- Découplage total : l'application Java et le conteneur peuvent être déployés sur des machines différentes sans aucune modification de code.
- Le conteneur est réutilisable par n'importe quel autre client (un script, une autre application, des tests automatisés).
- Le conteneur dispose d'un Dockerfile et d'un healthcheck natif Docker, ce qui en fait un composant d'infrastructure autonome et supervisable.

En cas d'indisponibilité du conteneur (non démarré, réseau coupé), l'application bascule automatiquement sur une heuristique locale dégradée (LocalStrengthEstimator) afin de ne jamais bloquer la fonctionnalité principale de génération. Ce mode de secours est clairement signalé à l'utilisateur dans l'affichage des résultats.

2.4 Sécurité de la génération

La génération utilise java.security.SecureRandom plutôt que java.util.Random, ce dernier étant un générateur pseudo-aléatoire prédictible (basé sur un algorithme linéaire congruentiel) et donc inadapté à un usage de sécurité. L'algorithme garantit également qu'au moins un caractère de chaque catégorie

sélectionnée est présent dans le mot de passe final, puis mélange l'ensemble pour éviter un ordre prévisible.

3. Guide d'installation

3.1 Prérequis

- Java 21 (JDK)
- Maven 3.9 ou supérieur
- Docker et Docker Compose

3.2 Construction et lancement du conteneur de validation

Depuis la racine du dépôt :

```
docker compose up -d --build
```

Vérification que le service répond :

```
curl http://localhost:5000/health
```

3.3 Compilation de l'application Java

```
cd MyPasswordCLI  
mvn clean package
```

Le jar exécutable est généré dans MyPasswordCLI/target/password-cli.jar.

3.4 Exécution

Mode interactif :

```
java -jar target/password-cli.jar
```

Mode paramètre, génération unique :

```
java -jar target/password-cli.jar --length=20 --upper --lower --digits --symbols
```

Mode rafale (10 mots de passe) :

```
java -jar target/password-cli.jar --length=16 --count=10
```

Sans audit Docker (estimation locale uniquement) :

```
java -jar target/password-cli.jar --length=12 --no-server
```

Aide complète :

```
java -jar target/password-cli.jar --help
```

3.5 Arrêt du conteneur

```
docker compose down
```